

Practical No.7

Aim:

To write a TypeScript program to demonstrate different methods to increase file size using Node.js file system operations.

Code:

```
// File Size Increase Program - TypeScript
// A program to increase file size by different methods

import * as fs from 'fs';
import * as path from 'path';
import * as readline from 'readline';

// Simple file size modifier class
class FileSizeModifier {
  // Get file size in bytes
  static getFileSize(filePath: string): number {
    if (!fs.existsSync(filePath)) {
      throw new Error(`File not found: ${filePath}`);
    }

    const stats = fs.statSync(filePath);
    return stats.size;
  }

  // Format size in bytes to human-readable format
  static formatSize(sizeBytes: number): string {
    const units = ['B', 'KB', 'MB', 'GB', 'TB'];
    let size = sizeBytes;

    for (const unit of units) {
      if (size < 1024) {
        return `${size.toFixed(2)} ${unit}`;
      }
      size /= 1024;
    }

    return `${size.toFixed(2)} PB`;
  }
}
```

```
// Increase file size by adding specified number of bytes
static increaseByBytes(filePath: string, bytesToAdd: number = 10): Oper
    const originalSize = this.getFileSize(filePath);

    // Open file in append mode and add bytes
    const buffer = Buffer.alloc(bytesToAdd, 0);
    fs.appendFileSync(filePath, buffer);

    const newSize = this.getFileSize(filePath);

    return {
        method: 'add_bytes',
        bytesAdded: bytesToAdd,
        originalSize,
        newSize,
        sizeIncrease: newSize - originalSize,
        originalSizeFormatted: this.formatSize(originalSize),
        newSizeFormatted: this.formatSize(newSize)
    };
}

// Increase file size by specified percentage
static increaseByPercentage(filePath: string, percentage: number = 10.0
    const originalSize = this.getFileSize(filePath);
    let bytesToAdd = Math.floor(originalSize * (percentage / 100.0));

    // Ensure at least 1 byte is added
    bytesToAdd = Math.max(1, bytesToAdd);

    const buffer = Buffer.alloc(bytesToAdd, 0);
    fs.appendFileSync(filePath, buffer);

    const newSize = this.getFileSize(filePath);

    return {
        method: 'percentage',
        percentage,
        bytesAdded: bytesToAdd,
        originalSize,
        newSize,
        sizeIncrease: newSize - originalSize,
        originalSizeFormatted: this.formatSize(originalSize),
        newSizeFormatted: this.formatSize(newSize)
    };
}

// Increase file size by multiplying it by specified factor
static increaseByMultiplier(filePath: string, multiplier: number = 10.0
```

```
const originalSize = this.getFileSize(filePath);
const targetSize = Math.floor(originalSize * multiplier);
let bytesToAdd = targetSize - originalSize;

// Ensure at least 1 byte is added
bytesToAdd = Math.max(1, bytesToAdd);

const buffer = Buffer.alloc(bytesToAdd, 0);
fs.appendFileSync(filePath, buffer);

const newSize = this.getFileSize(filePath);

return {
  method: 'multiplier',
  multiplier,
  bytesAdded: bytesToAdd,
  originalSize,
  newSize,
  sizeIncrease: newSize - originalSize,
  originalSizeFormatted: this.formatSize(originalSize),
  newSizeFormatted: this.formatSize(newSize)
};
}

// Create a test file with specified content
static createTestFile(filePath: string, content: string = 'Hello, World') {
  fs.writeFileSync(filePath, content);
  console.log(`Created test file: ${filePath}`);
  console.log(`Content: "${content}"`);
  console.log(`Initial size: ${this.formatSize(this.getFileSize(filePath))}`);
}

// Operation result interface
interface OperationResult {
  method: string;
  bytesAdded?: number;
  percentage?: number;
  multiplier?: number;
  originalSize: number;
  newSize: number;
  sizeIncrease: number;
  originalSizeFormatted: string;
  newSizeFormatted: string;
}

// Display the result of file size modification
function displayResult(result: OperationResult): void {
```

```

console.log('\n' + '='.repeat(70));
console.log('FILE SIZE MODIFICATION RESULT');
console.log('='.repeat(70));

console.log('\nMethod: ${result.method.toUpperCase().replace('_', ' ') }')

if (result.method === 'add_bytes') {
  console.log('Bytes added: ${result.bytesAdded}');
} else if (result.method === 'percentage') {
  console.log('Percentage: ${result.percentage}%');
} else if (result.method === 'multiplier') {
  console.log('Multiplier: ${result.multiplier}x');
}

console.log('\nOriginal size: ${result.originalSize} bytes (${result.or
console.log('New size:      ${result.newSize} bytes (${result.newSizeFo
console.log('Size increase: ${result.sizeIncrease} bytes (${FileSizeMod

console.log('\n File size increased successfully!');
}

// Method 1: Increase file size by 10 bytes (literal interpretation)
function method1Add10Bytes(): void {
  console.log('\n' + '='.repeat(70));
  console.log('METHOD 1: Increase File Size by 10 Bytes');
  console.log('='.repeat(70));
  console.log('\nInterpretation: Add exactly 10 bytes to the file');
  console.log('This is the most literal interpretation of \'increase by 1

// Create a test file
const testFile = 'test_method1.txt';
FileSizeModifier.createTestFile(testFile, 'Hello, World!\n');

// Increase by 10 bytes
const result = FileSizeModifier.increaseByBytes(testFile, 10);
displayResult(result);

// Clean up
fs.unlinkSync(testFile);
console.log('\nCleaned up test file: ${testFile}');
}

// Method 2: Increase file size by 10%
function method2IncreaseBy10Percent(): void {
  console.log('\n' + '='.repeat(70));
  console.log('METHOD 2: Increase File Size by 10%');
  console.log('='.repeat(70));
  console.log('\nInterpretation: Increase file size by 10 percent');

```

```
console.log('Example: 100 bytes → 110 bytes (10 bytes added)');

// Create a test file
const testFile = 'test_method2.txt';
FileSizeModifier.createTestFile(testFile, 'Hello, World!\n');

// Increase by 10%
const result = FileSizeModifier.increaseByPercentage(testFile, 10.0);
displayResult(result);

// Clean up
fs.unlinkSync(testFile);
console.log('\nCleaned up test file: ${testFile}');
}

// Method 3: Multiply file size by 10
function method3MultiplyBy10(): void {
  console.log('\n' + '='.repeat(70));
  console.log('METHOD 3: Multiply File Size by 10');
  console.log('=' .repeat(70));
  console.log('\nInterpretation: Make the file 10 times larger');
  console.log('Example: 100 bytes → 1000 bytes (900 bytes added)');

  // Create a test file
  const testFile = 'test_method3.txt';
  FileSizeModifier.createTestFile(testFile, 'Hello, World!\n');

  // Multiply by 10
  const result = FileSizeModifier.increaseByMultiplier(testFile, 10.0);
  displayResult(result);

  // Clean up
  fs.unlinkSync(testFile);
  console.log('\nCleaned up test file: ${testFile}');
}

// Create interface for readline
function createInterface(): readline.Interface {
  return readline.createInterface({
    input: process.stdin,
    output: process.stdout
  });
}

// Promisify readline question
function question(rl: readline.Interface, query: string): Promise<string>
return new Promise(resolve => {
  rl.question(query, (answer: string) => {
```

```
        resolve(answer.trim());
    });
});
}

// Interactive mode for user to modify any file
async function interactiveMode(): Promise<void> {
    console.log('\n' + '='.repeat(70));
    console.log('INTERACTIVE MODE');
    console.log('='.repeat(70));

    const rl = createInterface();

    const filePath = await question(rl, '\nEnter file path: ');

    if (!fs.existsSync(filePath)) {
        console.log(`Error: File not found: ${filePath}`);
        rl.close();
        return;
    }

    const originalSize = FileSizeModifier.getFileSize(filePath);
    console.log(`\nCurrent file size: ${originalSize} bytes (${FileSizeModi

    console.log('\nChoose modification method:');
    console.log('1. Add 10 bytes (default)');
    console.log('2. Increase by 10%');
    console.log('3. Multiply by 10');
    console.log('4. Custom amount');

    const choice = await question(rl, '\nEnter choice (1-4): ') || '1';

    let result: OperationResult;

    if (choice === '1') {
        result = FileSizeModifier.increaseByBytes(filePath, 10);
        displayResult(result);
    } else if (choice === '2') {
        result = FileSizeModifier.increaseByPercentage(filePath, 10.0);
        displayResult(result);
    } else if (choice === '3') {
        result = FileSizeModifier.increaseByMultiplier(filePath, 10.0);
        displayResult(result);
    } else if (choice === '4') {
        const amountStr = await question(rl, 'Enter amount: ');
        const amount = parseFloat(amountStr);

        if (isNaN(amount)) {
```

```
        console.log('Error: Invalid amount');
        rl.close();
        return;
    }

    const subChoice = await question(rl, 'Add bytes (b), percentage (p),

    if (subChoice === 'p') {
        result = FileSizeModifier.increaseByPercentage(filePath, amount);
        displayResult(result);
    } else if (subChoice === 'm') {
        result = FileSizeModifier.increaseByMultiplier(filePath, amount);
        displayResult(result);
    } else {
        result = FileSizeModifier.increaseByBytes(filePath, Math.floor(amou
        displayResult(result);
    }
    } else {
        console.log('Error: Invalid choice');
    }

    rl.close();
}

// Main function
async function main(): Promise<void> {
    console.log('=' .repeat(70));
    console.log('FILE SIZE INCREASE PROGRAM');
    console.log('=' .repeat(70));
    console.log('\nThis program demonstrates different methods to increase

    // Run all three methods
    method1Add10Bytes();
    method2IncreaseBy10Percent();
    method3MultiplyBy10();

    // Uncomment to enable interactive mode:
    // await interactiveMode();
}

// Run if executed directly
main().catch(console.error);
```

Output:

practical7-output.png